

*

Stack Demo :-

1st top = -1

then ++ top = 0

expression :- { { C } } }

C	2	{ { {
{	1	{ { C → Push
{	0	} } } → POP

so after pushing { { C POP is occurs from index 2 then 1 and then 0.

while popping element it checks that at index 2 : C = }

-1 - 1 : { = {

0 : { = {

if it match it gives output balanced if opening & closing bracket are not matching then it not balanced

*

Stack Demo - Generic :-

It work same as stack Demo only difference is it accept different type of value like char, int & string.

in char we gives expression like { { } } and for int & string there are hard coded value so for that top value get pop we which are 20 for int & word for string.

* StackArrayInt Demo :-

at first we created array of n size
suppose : $N = 4$

so $top = -1$

then $top = 0$ by $(++top)$

$\therefore$ index 0 $\rightarrow 10$

index 1 $\rightarrow 20$

-1 - 2 $\rightarrow 30$

-1 - 3 $\rightarrow 40$

$\therefore$ value = 10, 20, 30, 40

display show elements (top to bottom) =
40, 30, 20, 10

peek() gives us top element which is 40

pop() removes the top element 40

if we try to push 50 & 60 then 50 get push in stack but not 60 it gives message i.e. stack overflow.

* Queue Demo :-

capacity : max = 5

front =

rear = -1

queue = [0, 0, 0, 0, 0]

• enqueue = 10

IF (front == -1) {

front = 0

} true

* StackArrayIntDemo :-

Date _____
Page _____

at first we created array of n size
suppose : $N = 4$

so $top = -1$

then $top = 0$ by $(++top)$

$\therefore$ index 0 $\rightarrow 10$

index 1 $\rightarrow 20$

-1 - 2 $\rightarrow 30$

-1 - 3 $\rightarrow 40$

$\therefore$ value = 10, 20, 30, 40

display show elements (top to bottom) =
40, 30, 20, 10

peek() gives us top element which is 40

pop() removes the top element 40

if we try to push 50 & 60 then 50 get push in stack but not 60 it gives message i.e. stack overflow.

* QueueDemo :-

capacity : max = 5

front = -1

rear = -1

queue = [0, 0, 0, 0, 0]

enqueue = 10

IF (front == -1) {

front = 0

} true

$\therefore \text{front} = 0$
 $\Delta \text{ queue} [++\text{rear}] = \text{rear} : 0$

$\therefore \text{front} = 0, \text{rear} = 0$
 $\text{array} = [10, 0, 0, 0, 0]$

• enqueue 20, 30, & 40

20: rear increments to 1

30: rear increments to 2

40: rear increments to 3

$\therefore \text{front} = 0, \text{rear} = 3$

$\text{array} = [10, 20, 30, 40, 0]$

• display queue :

queue elements : 10, 20, 30, 40
 index : 0, 1, 2, 3

• dequeue

$\text{int removed} = \text{queue} [\text{front}++] ;$

on deque
 do front
 increase
 to 4

$\therefore \text{front} = 0$

10 removed from queue

$\therefore \text{front now} = 1$ by $[\text{front}++]$

20 removed from queue

$\therefore \text{front} = 2$

30 removed from queue

$\therefore \text{front} = 3$

40 removed from queue

on deque if check (isEmpty) $\rightarrow$ true

gives output : queue underflow

∴ front = -1
rear = -1

b) If (front > rear) {
front = -1;
rear = -1;
}

473

as on deque front = 3

rear = 3

@demoGeneric :-

It same as queueDemo but it accept different type of values like char, string we have hardcode value.

```
@demoGeneric <String> textQueue = new  
@demoGeneric <>();
```

∴ front = -1

rear = -1

array = [0, 0, 0, ...]

queue : "A"

front == -1 → true

∴ front = 0 updates

queue[++rear] = 'A' rear increment to 0

∴ front = 0 rear = 0

array = ["A", 0, 0, ...]

∴ front = -1
rear = -1

b) IF (front > rear) {
 front = -1;
 rear = -1;
}

473

as on deque front = 3 rear = 3

demoGeneric :-

It same as queueDemo but it accept different type of values like char, string.

we have hardcoded value.

```
demoGeneric <String> textQueue = new  
demoGeneric <>();
```

∴ front = -1

rear = -1

array = [0, 0, 0, ...]

queue : "A"

front == -1 → true

∴ front = 0 updates

queue[++rear] = 'A' rear increment to 0

∴ front = 0 rear = 0

array = ["A", 0, 0, ...]

queue B :

front = -1 false $\therefore$ front = 0
queue[++rear]

$\therefore$ front = 0 rear = 1

array = ["A", "B", 0, ...]

dequeue :-

T removed = (T) queue[front++] ;

remove = "A" $\therefore$ front increment
to 1

on deque it remove "B" $\therefore$ front = 2

$\therefore$ front = 2 > rear = 1

$\therefore$ front = -1

rear = -1

using if (front > rear) {

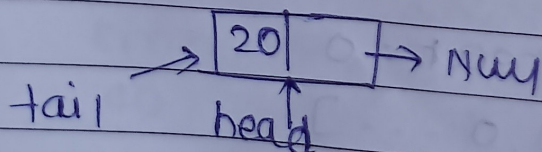
front = -1 ;

rear = -1 ;

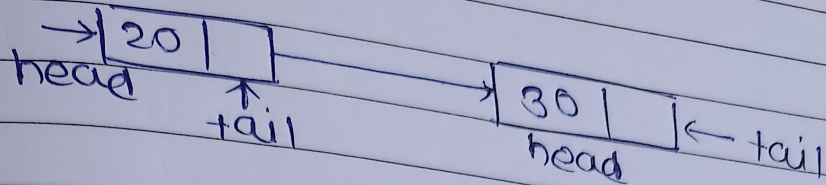
}

* single linked list :-

add last 20 :-



addlast 30:



addfirst 10:-

